

pRTMNSGA-III: A novel Multi-Objective Algorithm for QoS-Aware Multi-Cloud IoT Service Selection

Ahmed ZEBOUCHI^{1*} and Youcef AKLOUF¹

¹RIIMA, USTHB, BP 32 El-Alia, Bab-Ezzouar, 16111, Algiers, Algeria.

*Corresponding author(s). E-mail(s): azebouchi@usthb.dz;
Contributing authors: yaklouf@usthb.dz;

Abstract

The Internet of Things (IoT) is an emerging technology expected to play a significant role. The integration of IoT with Cloud Computing (CC) has enabled the creation of large-scale networks of interconnected smart devices and services. To provide optimal Quality of Service (QoS) for users, it is necessary to address the conflicting requirements of IoT services. The service selection problem is considered as NP-hard, and therefore requires using metaheuristic algorithms for efficient resolution. This paper presents a novel hybrid multi-objective metaheuristic, pRTMNSGA-III, which combines the strengths of RNSGA-III and TMNSGA-III to generate solutions that meet user preferences and eliminate unfavorable ones. To further optimize computational time, a parallel solution evaluation approach is employed. In addition, a more effective fuzzy membership function is proposed to select the best solution based on the requester's preferred QoS. The proposed algorithm is evaluated on multiple datasets, and the results demonstrate its superiority over existing state-of-the-art algorithms, including RNSGA-III, TMNSGA-III, NSGAIII, pNSGAII, NSGA-II and NSPSO. Experimental results demonstrate that pRTMNSGA-III outperforms existing algorithms by up to 37% in terms of the number of non-dominated solutions generated.

Keywords: Artificial Intelligence for IoT, Multi-cloud, Service selection, Parallel hybrid metaheuristic, Fuzzy decision maker

1 Introduction

The Internet of Things (IoT) is a rapidly evolving technology comprising interconnected devices linked to the Internet for information exchange. Until recently, only computers, tablets, and phones could connect, but IoT extends connectivity to virtually all appliances, enabling remote monitoring. The core idea is to create an intelligent environment that's easy to monitor. IoT adoption is growing rapidly, with 13.1 billion connected devices presently and an expected 30.1 billion by 2030 [1]. As the number of IoT-connected devices rises, the importance of Cloud Computing (CC) in IoT development becomes evident.

The cloud represents a virtual resource pool accessible via the Internet, using large data centers and powerful servers to host web services. Its abundant resources greatly enhance IoT functionality and connectivity. While a single cloud might not consistently fulfill all service demands, especially complex ones, a more effective approach involves multi-cloud utilization, cloud brokering, and various Quality of Service (QoS) levels [2]. Multi-cloud combines public and private clouds, while cloud brokering bridges them for simpler data management and user request fulfillment across multiple clouds.

To address specific QoS needs that are often overlooked, some cloud service providers establish federations of several clouds to ensure customer satisfaction while delivering high-quality service. The challenge of cloud service selection and composition is a well-established NP-hard optimization problem [3], signifying that finding an optimal solution within a practical timeframe is computationally unfeasible, especially for large-scale scenarios. To address this complexity, various metaheuristic algorithms, including genetic algorithms, have been proposed to tackle the problem. Furthermore, when users specify their preferences for IoT services, the challenge intensifies due to conflicting objectives, such as the need to minimize costs while simultaneously optimizing execution time. In such cases, traditional metaheuristics may fall short. This problem is categorized as a Multi-objective NP-Hard problem, necessitating the application of Multi-objective metaheuristics to effectively navigate the intricacies of these conflicting objectives and deliver satisfactory solutions.

Metaheuristics, including genetic algorithms, are an evolving field intended to assist researchers in resolving optimization problems efficiently. The literature highlights the lack of multi-criteria service selection mechanisms in multi-cloud environments [4]. Our paper makes two significant contributions to address the multi-cloud IoT service selection problem while considering Quality of Service (QoS) preferences.

Firstly, we leverage two of the most effective resolution algorithms, RNSGA-III and TMNSGA-III, and introduce a new hybrid algorithm, pRTMNSGA-III. This new algorithm combines the strengths of both methods and extends them to a parallel version for improved performance.

Secondly, we integrate a suitable type of fuzzy membership function into the solution selection process of the multi-objective algorithm. This approach

facilitates the identification of the best solution from the solution set generated by the multi-objective algorithm.

Throughout our research, we collected and compiled all Quality of Service (QoS) factors considered in service selection. This allowed us to compare existing problem-solving approaches with our proposed algorithm and ensure the optimal Service Level Agreement (SLA) was achieved.

The effectiveness of our approach was evaluated through a series of experiments on multiple datasets, each with varying service requests, numbers of services, and numbers of clouds. The results demonstrate that pRTMNSGA-III outperforms comparative methods, delivering superior outcomes, as shown in the experimental section.

The rest of this paper is organized as follows. Section 2 discusses related work. Section 3 is about problem formulation. In Section 4, we will explaining our proposed approach. Section 5 is a discussion of the experiment. Finally, in Section 6, the conclusion and some perspectives are given.

2 Related works

IoT services in the multi-cloud face difficulties locating and selecting suitable services while achieving high efficiency and optimization. In order to solve this issue, one must obtain a set of optimal Pareto solutions that meet the specifications of convergence to the true Pareto front and uniform diversity [5].

This section reviews the works that implemented similar solutions to solve the IoT quality of service selection problem.

This paper introduces a multicriteria optimization model for selecting cloud service providers in multicloud environments, considering factors such as cost, performance, reliability, and security while adhering to customer budget constraints [6]. However, it does not cover all possible selection criteria and lacks comparative analyses with other models, making it less practical for large-scale problems.

In a similar vein, [7] presents a dynamic service selection paradigm for multiple manufacturing clouds, utilizing IoT and big data analytics to optimize service selection for cost reduction while meeting quality of service (QoS) requirements. Critics argue that its lack of comparative analyses, scalability discussions, and security/privacy implications hinder a comprehensive understanding of its practical applicability.

Addressing cloud-based IoT applications, [8] proposes the Energy-Aware Service Composition (EASC) algorithm, which considers both energy consumption and QoS metrics during service composition. While showing promise in reducing energy consumption, this work needs further comparative analyses, scalability discussions, and considerations for real-world validation.

Similarly, [9] delves into the realm of cloud manufacturing and presents a framework for dynamic service selection (SS) across multiple manufacturing clouds (MCs). The framework leverages the potential of the Internet of Things

(IoT) and Big Data to optimize SS in complex manufacturing scenarios. Critics point out that this paper lacks clear positioning, a structured literature review, and a comparative analysis with other paradigms.

Introducing the Energy-Centric and QoS-Aware Service Selection Algorithm (EQSA) for IoT, [10] the paper presents an innovative energy-centric and quality of service (QoS)-aware service selection algorithm (EQSA) for the Internet of Things (IoT). This algorithm strives to harmonize energy efficiency and QoS requirements in IoT services composition. It employs a lexicographic optimization strategy and a QoS constraints relaxation technique to preselect services based on QoS and then selects the most suitable services with a focus on energy profiles, QoS attributes, and user preferences, following the principles of Pareto dominance. The absence of real-world validation, detailed scalability discussions, and the failure to address IoT data complexities are highlighted as concerns

In [11], the paper authored by Khadir et al, presents a genetic algorithm (GA)-based approach for selecting IoT services that are aware of fluctuating Quality of Service (QoS). This approach considers both the global QoS constraints of the IoT application and the local QoS constraints of individual IoT services. Moreover, it is noteworthy for its distribution, making it particularly suitable for scaling to large-scale IoT systems. Otherwise the study lacks a comparative analysis with other approaches, and does not thoroughly discuss simulation limitations.

the paper [12] authored by Wang and Lu introduces a novel hybrid metaheuristic algorithm (HMHA) for enhancing Quality of Service (QoS)-aware service discovery and selection in cloud-edge computing environments within the Internet of Things (IoT) domain. This innovative approach combines a grey wolf optimizer (GWO) with a genetic algorithm (GA) to optimize service selection with the dual objective of meeting QoS requirements while minimizing operational costs. still faces concerns about the lack of comparative analyses with other algorithms and scalability discussions.

Moreover, [13] authored by Mohamed and Abdelsalam (2018) introduces a novel cuckoo-inspired algorithm (CIA) for multi-cloud service composition in the context of Internet of Things (IoT) applications. The CIA algorithm, inspired by the behaviors of cuckoo birds, is designed to efficiently find composite services that meet user requests within a multi-cloud environment. Critics mention the lack of use of metaheuristics in a proper way as in definition, and a lack of a comprehensive comparative analysis and scalability discussions,

the paper [14] addresses the challenges of cloud brokering in the Internet of Things (IoT) by proposing a multiobjective optimization (MOO) framework. This framework is designed to maximize the profit of cloud brokers while minimizing response time and energy consumption in the context of connected IoT systems. Notably, the authors employ a multiobjective particle swarm optimization (MOPSO) algorithm to solve this complex optimization problem, and they provide a comparative analysis against genetic and random search algorithms, with MOPSO emerging as the superior choice for multiobjective

optimization, the study shows the absence of comparative analyses with other MOO frameworks, discussions on scalability, and considerations for security and privacy.

Lastly, [15] Liu et al introduces a novel approach to optimizing Inter-Cloud service composition. Their hybrid multi-objective evolutionary algorithm (MOEA), LS-NSGA-II-DE, is designed to address the complex challenges of service composition in cloud environments. The algorithm combines differential evolution (DE) with non-dominated sorting genetic algorithm (NSGA-II), aiming to enhance convergence and diversity. Additionally, a local search (LS) method is incorporated to improve the distribution of non-dominated solutions. The paper evaluates this MOEA through simulation experiments and establishes its superiority over other inter-cloud service composition algorithms regarding the number of generated solutions, feasibility, and running time. However, the study lacks a comprehensive comparative analysis and discussions on scalability

In [16], A Survey on the metaheuristic-based resolution methods for multi-cloud IoT service selection evaluates 18 MO Algorithms considering 8 of the most targeted Multi-Cloud QoS for IoT. It shows the best algorithms to select IoT services across various Cloud Providers (CPs) to achieve the best performances on the selected QoS. From the experiment's findings of [16], it came out that the pNSGA-II [17] algorithm performs better than other approaches and is the most suitable solution for the multi-cloud IoT service selection problem among the 18 compared algorithms. Then, pNSGA-II was improved to NSGA-III [18]. But, since this last presented some drawbacks in some use-cases, it has also been improved to two distinct versions called RNSGA-III [19] and TMNSGA-III [20], which outperformed the original NSGA-III in terms of solution quality and computational efficiency.

In our research, we place a strong emphasis on addressing the critical factors of multicloud environments and multi-objective optimization, which have been identified as significant challenges in the field. We take into consideration the diverse criteria and complexities associated with multicloud setups, aiming to optimize the service selection process for users with varying requirements. To accomplish this, we have meticulously designed our pRTMNSGA-III algorithm, which leverages the strengths of two leading algorithms, RNSGA-III and TMNSGA-III, and implements a parallel evaluation of solutions to enhance efficiency. Moreover, the selection of the best solution for service requesters is refined through a more appropriate fuzzy decision-making approach.

Our innovative approach builds upon the advancements made in previous studies, as highlighted in the comparison provided in Table 1. The pRTMNSGA-III algorithm's unique features make it well-suited to tackle the multidimensional aspects of the multi-objective problem at hand, contributing to its superior performance compared to existing algorithms.

Study	Multicloud Environment	Multicriteria Model	Comparative Analysis	Scalability
Mohamed, A.M. et al[6]	Yes	- Does not account for all criteria	- Lacks comparative analyses	- Less practical for large-scale problems
Yang, L. et al[7]	Yes	- Not discussed	- Lack of comparative analyses	- Scalability concerns
Baker, T. et al[8]	No	- Not discussed	- Needs comparative analyses	- Scalability discussions
Chen, Y. et al[9]	Yes	- Not discussed	- Lack of comparative analyses	- Scalability concerns
Ait Aissa, M. et al[10]	No	- Not discussed	- Lacks real-world validation, comparative analyses	- Detailed scalability discussions
Benouiza, A. et al[11]	No	- Not discussed	- Needs comparative analyses	- Not discussed
Wang, R. et al[12]	Yes	- Not discussed	- Lack of comparative analyses	- Scalability concerns
Mohamed, A.M. et al[13]	Yes	- Not discussed	- Needs comparative analyses	- Scalability concerns
Kumrai, T., et al[14]	No	- Not discussed	- Lack of comparative analyses	- Not discussed
Liu, J. et al[15]	Yes	- Not discussed	- Needs comparative analyses	- Scalability discussions

Table 1: Comparison of Selected Research Papers

3 Problem Formulation

In IoT service selection, we optimize multi-cloud solutions using quality of services (QoS) and multi-objective metaheuristics. The challenge is aligning services with user specifications amid conflicting QoS attributes.

-Dominance: Dominance is determined by comparing objective functions:

1. If $f_i(x_1) \leq f_i(x_2)$ for all i in $\vec{f}$.
2. If there exists an i such that $f_i(\vec{x}) < f_i(\vec{y})$.

The optimal solution forms the Pareto front, a set of solutions, with the fittest ones closest to it, and are used to create new solutions [21].

Pareto Front: The Pareto Front contains non-dominated solutions, ensuring no improvement in one objective sacrifices another [21].

Quality of Service (QoS): We seek the best services based on user preferences expressed in service requests, focusing on recent QoS studies [21]. We performed a comprehensive analysis of the most crucial QoSs for IoT services

in a multi-cloud environment based on the study by [16]. We identified the key QoS that are most relevant to our research and aligned our choices with existing research to provide a strong rationale for our selection.

3.1 Solution Encoding

In this article, we represent service composition as an array where each cell encodes a service instance. For instance, Table 3 shows a composite service with 7 service classes.

3.2 QoS Model

We evaluate solution quality using Quality of Service (QoS) and aggregate QoS measurements, as shown in Table 2. Optimizing service composition aims to meet user QoS requirements across a vast search space. QoS criteria are normalized to a range of $[0, 1]$ using Equation 1:

$$Q_{x_{im}} = \frac{Q_{x_{im}} - Q_{\min(m)}}{Q_{\max(m)} - Q_{\min(m)}} \quad (1)$$

The optimization process balances positive and negative criteria, transforming all goals into minimization objectives.

4 Multi-Cloud MO IoT Service Selection

In a study by Zebouchi and Aklouf [16], various multi-objective metaheuristics were evaluated to identify the most efficient algorithm in terms of solution quality and execution time. The study found that NSGA-II ranked highest among all tested algorithms. NSGA-II was further enhanced to create NSGA-III, which was subsequently improved in several studies, including TMNSGA-III, RNSGA-III, U-NSGA-III, A-NSGA-III, and tDOM-NSGA-III. We focused on two of these algorithms, RNSGA-III and TMNSGA-III, due to their alignment with user preferences and support for a stopping criterion. We also introduced a hybrid multi-objective algorithm, pRTMNSGA-III, by combining RNSGA-III and TMNSGA-III to harness their enhancements. In the following sections, we will describe RNSGA-III and TMNSGA-III, along with the hybridization we developed.

4.1 RNSGA-III

RNSGA-III is a reference point-based multi-objective metaheuristic algorithm that builds upon NSGA-III. It exhibits enhanced capabilities in solving objective problems and is sometimes proven to be faster than NSGA-III. The working principle of RNSGA-III is illustrated through its successful application to numerous many-objective optimization tests and real-world problems, as demonstrated by Vesikar, Deb, and Blank in their study [24]. RNSGA-III extends the NSGA-III procedure by introducing a novel reference point generation method based on user-supplied aspiration points, while retaining the

Table 2: Description of QoS in IoT.

QoS Name	QoS Description	QoS Formula
Availability	The Availability [22] $A(S_i^j)$ of service i in cloud j is calculated as $A = t/t_s$ where t and t_s represent the uptime and the total time of service. As the value of A approaches 1, availability increases.	$A_{global} = \sum_{i=1}^N A(S_i^j)$
Cost	After the apparent rise in CP numbers, The most recent studies concentrate on lowering cloud prices since CP offers a vast array of services, some of which have evolved into essential components of the QoS [22]. The overall cost is estimated through the accumulation of all the costs $C(S_i^j)$ of service i from provider j .	$C_{global} = \sum_{i=1}^N C(S_i^j)$
Energy Consumption	The amount of energy needed to run IoT services is crucial for measuring the quality of service, and it is regarded as a significant problem in cloud computing. Therefore, service providers strive to finish the job efficiently with the least energy. Assuming $E(S_i^j)$ is the energy i consumption used by service i to perform the job by cloud provider j and N is the number of services included in the service composition [23].	$E_{global} = \sum_{i=1}^N E(S_i^j)$
Profit	Profit [22] is a monetary reward that cloud service providers can sometimes earn. Also when identifying the ideal solution the broker is expected to profit. Therefore, the broker's profit is regarded as a secondary objective. It is calculated based on the difference between the consumer price $P(S_i^j)$ of service i of the service i provider j and the service provider value $C(S_i^j)$ of service i provided by cloud j	$P_{global} = \sum_{i=1}^N P(S_i^j) - C(S_i^j).$
Number of clouds involved in the composition	The number of clouds used in the service composition, plus the many services from multiple clouds, increases inter-cloud communication, which raises energy consumption and communication costs. (α : boolean; 0 if Cloud already added, else 1).	$\theta := \sum_{i=1}^{\text{composition}} (\alpha \cdot \text{Cloud}(S_i))$
Number of clouds involved in composition plan	Clouds already have a predefined composition plan list that is already developed and optimized inciting composition to use a minimal number of cloud composition plans enhancing the response time and lowering the costs. (α : boolean; 0 if C plan already added, else 1).	$\gamma := \sum_{i=1}^{\text{composition}} (\alpha \cdot \text{C_plan}(S_i))$
Reliability	The reliability determines the assertion level lacking all sorts of hardware or software issues with cloud services. It is the total sum of the $R(S_i^j)$ reliability of each service i in cloud j . As the value of R approaches 1, the reliability increases.	$R_{global} = \sum_{i=1}^N R(S_i^j)$
Response Time	Response time is the amount of time between sending a request and waiting for the service to respond. It is calculated by summing up the latency difference between customer request time and service provider response time $L(S_i^j)$ and the execution time $ET(S_i^j)$ of service i in cloud j .	$RT_{global} = \sum_{i=1}^N L(S_i^j) + ET(S_i^j)$

same genetic operators and survival selection process. This extension aims to achieve a more uniform emphasis on generating solutions for all aspiration

S1	S2	S3	S4	S5	S6	S7
101	32	12	65	195	26	21

Table 3: Example of a Solution

points. Research has shown that the RNSGA-III method, as evidenced in [24], is beneficial for comprehensive optimization and decision-making, particularly when preference information is available in the objective space.

In RNSGA-III, reference lines connect the reference points to the origin of the multidimensional space, serving as a linkage between the generated solutions and the reference points. Reference points primarily contribute to maintaining diversity within the solution population. In contrast to NSGA-III, where reference point selection is accomplished using the Das and Dennis approach, RNSGA-III chooses a suitable set of uniformly spaced reference points on a unit hyperplane (referred to as H reference points). Additionally, it establishes reference directions (H) in the positive objective coordinate space by connecting each reference point to the origin. Before creating Das and Dennis points on a unit hyperplane, RNSGA-III calculates ideal point vectors and unit hyperplane intercepts at each normalized reference point. These points are subsequently scaled down by a factor and shifted in the direction of the unit hyperplane. This process is repeated in each generation. Following population normalization, the RNSGA-III procedure associates the output with reference points. The step-by-step process of RNSGA-III is detailed in Algorithm 1.

Algorithm 1 Algorithm RNSGA-III

Objective functions (Obj), constraints (Constr), Aspiration points (Apts),
d, N, tmax, pc, pm, μ
pop;F = NonDominatedSorting(pop)
pop = NormalisePopulation(pop)
pop = AssociateToRefPoint(pop;F)
pop = SortPopulation(pop)
for it = 1 to t_{max} **do**
 popm = Mutate(pop; pm; μ ;s)
 popc = Crossover(pop; pc)
 $pop_{it} = pop \cup pop_c \cup pop_m$
 $pop_{it};Fit = \text{NonDominatedSorting}(pop_{it})$
 $pop_{it} = \text{AssociateToRefPoint}(pop_{it};F)$
 $pop_{it} = \text{SortPopulation}(pop_{it})$
 pop = popit(1 : N)
 pop = NonDominatedSorting(pop)
 pop = NormalisePopulation($pop_{it} + \text{Apts}$)
 $Z_a = \text{RNSGAIII_referencepoints_procedure}()$
 pop = AssociateToRefPoint(pop; Z_a)
 pop = SortPopulation(pop)
end for

Advantages of RNSGA-III

RNSGA-III's main advantage lies in its focus on user objective preferences, enabling efficient analysis and validation of specific sections of the trade-off frontier generated by Multi-objective optimization (EMO). For instance, it can investigate gaps or points of interest within the preferred segment of the trade-off frontier, boosting confidence in decision-making for five-objective problems. RNSGA-III emphasizes targeted areas of the Pareto optimal front, potentially surpassing the original NSGA-III.

In RNSGA-III, solutions connect to reference points through reference lines, replacing NSGA-III's crowding distance for maintaining solution diversity.

Downsides of RNSGA-III

However, RNSGA-III has a notable drawback: it can inefficiently allocate computational resources. Non-functional reference points may be discovered and relocated near existing reference points, which have already identified non-dominated solutions.

4.2 TMNSGA-III

NSGA-II and NSGA-III faced issues of being slow, inefficient, and problem-specific. TMNSGA-III, introduced by Viviane de Buck, addressed these problems by incorporating trade-off functions (T) and a specialized stopping criterion (M). The Mean Ideal Distance (MID) signals unnecessary iterations once a solution population converges to the Pareto front, alongside the Fraction of Pareto-optimal Solutions (FPOS).

TMNSGA-III's enhancements include post-assignment trade-off functions and the removal of redundant diversity-enabling and sorting functions. It's evaluated for its reduced computation time, improved solution diversity, and optimized solution density across trade-off areas [20]. Algorithm 2 provides an overview of the TMNSGA-III mechanism.

Advantages of TMNSGA-III

Prior NSGA-III variants had a simple stopping criterion, often wasting resources with a fixed iteration limit. TMNSGA-III introduced a trade-off function, prioritizing high trade-offs and enabling more efficient selection.

TMNSGA-III benefits include:

1. Reduced Computation Time.
2. Enhanced Solution Diversity.
3. Optimized Pareto Front Density.

This improved efficiency stems from a problem-specific stopping criterion and trade-off functions

Downside of TMNSGA-III

The primary drawback of a posteriori decision-making lies in its demand for generating numerous solutions, a significant portion of which is ultimately discarded during the filtering process. This simultaneous discarding implies a

Algorithm 2 Algorithm TMNSGA-III

```

Require: Aspiration points (Apts), d, N,  $t_{max}$ , pc, pm,  $\Delta t$ ,  $\Delta r$ ,  $\Delta MID$ 
pop;F = NonDominatedSorting(pop)
pop = NormalisePopulation(pop)
pop = AssociateToRefPoint(pop;F)
pop = SortPopulation(pop)
for it = 1 to  $t_{max}$  do
    popm = Mutate(pop; pm;s)
    popc = Crossover(pop; pc)
     $pop_{it} = pop \cup pop_c \cup pop_m$ 
     $pop_{it};F_{it} = \text{NonDominatedSorting}(pop_{it})$ 
     $pop_{it} = \text{AssociateToRefPoint}(pop_{it};F)$ 
     $pop_{it} = \text{CalcTradeOff}(pop_{it}, \Delta t, \Delta r, F_{it})$ 
     $pop_{it} = \text{SortPopulation}(pop_{it})$ 
    pop = popit(1 : N)
    pop = NonDominatedSorting(pop)
     $MID_{it} = \text{CalculateMID}(pop)$ 
     $FPOS_{it} = |F_1|/N$ 
    if  $|MID_{it} - MID_{it-1}| \leq \Delta MID$  and  $FPOS_{it} = 1$  then
        break;
    end if
end for

```

substantial computational overhead to produce superfluous solutions. Hashem et al. [25] introduced a divide-and-conquer scheme, empowering deterministic algorithms like Normal Boundary Intersection (NBI) to generate solutions meeting the minimal required trade-off values Δt and distribution Δr . Solutions are generated at a rate of one per iteration through a recursive scheme. If a generated solution no longer complies with the minimal trade-off Δt or distribution Δr , the algorithm is either halted or urged to cease exploration in the corresponding region of the Pareto front.

The trade-off values Δt and distribution Δr of solutions also serve as an additional stopping criterion. The generation of multiple solutions per iteration by stochastic algorithms renders the divide-and-conquer scheme introduced by Hashem et al. [25] unsuitable for evolutionary algorithms such as NSGA-II and NSGA-III.

Table 4 provides a concise comparison between two multi-objective optimization algorithms, RNSGA-III and TMNSGA-III. It highlights their key features, efficiency, as well as notable strengths and weaknesses, aiding in the selection of the most suitable algorithm for specific optimization tasks.

4.3 Proposed Approach

The proposed approach comprises three primary components. First, entities manage multi-cloud service information and prepare input for the solution.

Criteria	RNSGA-III	TMNSGA-III
Working Principle	Reference point-based NSGA-III metaheuristic	Incorporates trade-off functions and a novel stopping criterion
Efficiency	Faster than NSGA-III	Addresses inefficiencies of NSGA-II and NSGA-III, improves speed, and adds stopping criterion
Objective Preference	Focuses on user objective preferences	Introduces trade-off functions for preference-based optimization
Diversity Maintenance	Uses reference points to maintain diversity	eliminates diversity-enabling functions
Stopping Criterion	Lacks a problem-specific stopping criterion	Implements a stopping criterion based on MID and FPOS to reduce computational waste
Computational Efficiency	Offers computational efficiency for trade-off analysis	Prioritizes high-value solutions over low-value ones, reducing computational effort
Solution Density in Trade-Off Areas	Maintains uniform solution density	Emphasizes high-density in high trade-off areas and low-density in low trade-off areas
Downsides	Waste of computational effort due to non-functional reference points	Generates many solutions that may be discarded, leading to inefficiency
Advantages	Effective for preference-based optimization	Significantly reduces computation time, enhances solution diversity, and optimizes solution density in trade-off areas

Table 4: Comparative Analysis of RNSGA-III and TMNSGA-III

Second, the algorithm processes the information to determine the optimal front. Finally, the third step involves selecting a solution based on user preferences and delivering it to the service requester. The entire process is illustrated in Figure 2.

4.3.1 Service Request to Input Algorithm

In the cloud broker, the unit known as the cloud crawler continuously collects available IoT services from cloud service providers and stores service information, including QoS, in the database. The cloud broker then receives a service request and passes it through various units, including service identification and service composition, to break down the request into a set of unique services. Subsequently, the broker checks whether the service request has been previously fulfilled. If so, it verifies the availability of all services and responds to the service requester. If not, it utilizes the multi-objective algorithm to search for a solution.

4.3.2 Proposed pRTMNSGA-III

The algorithms, RNSGA-III and TMNSGA-III, have significantly improved the resolution of NSGA-III's drawbacks in various objective problems, resulting in enhanced quality and execution time. This paper marks the first development of a hybridization between these two algorithms. This hybridization leverages the strengths of each algorithm to achieve higher efficiency. Furthermore, the algorithm incorporates evaluation parallelism and integrates fuzzy theory decision-making, introducing a novel approach known as pRTMNSGA-III.

Algorithm 3 operates based on user preferences from the outset, making it highly optimized and well-suited for multi-cloud IoT service selection, which often involves a vast number of provided services. It features a trade-off function for filtering solutions, retaining the most promising ones, along with an additional stopping criterion to prevent computational time wastage.

Algorithm 3 Algorithm pRTMNSGA-III

```

1: Require :  $\mu$ , Aspiration points (Apts), d, N,  $t_{max}$ , pc, pm,  $\Delta t$ ,  $\Delta r$ ,  $\Delta MID$ 
2: pop;F = NonDominatedSorting(pop)
3: pop = NormalisePopulation(pop)
4: pop = AssociateToRefPoint(pop;F)
5: pop = SortPopulation(pop)
6: for  $it = 1$  to  $t_{max}$  do
7:   popm = Mutate(pop; pm; $\mu$ ;s)
8:   popc = Crossover(pop; pc)
9:    $pop_{it} = pop \cup pop_c \cup pop_m$ 
10:   $pop_{it}$ ;Fit = NonDominatedSorting( $pop_{it}$ )
11:   $pop_{it} = AssociateToRefPoint(pop_{it}; F)$ 
12:   $pop_{it} = CalcTradeOff(pop_{it}, \Delta t, \Delta r, F_{it})$ 
13:   $pop_{it} = SortPopulation(pop_{it})$ 
14:  pop =  $pop_{it}(1 : N)$ 
15:  pop = NonDominatedSorting(pop)
16:  pop = NormalisePopulation( $pop_{it}$  + Apts)
17:   $Z_a = RNSGAIII\_referencepoints\_procedure()$ 
18:  pop = AssociateToRefPoint(pop;  $Z_a$ )
19:  pop = NonDominatedSorting(pop)
20:   $MID_{it} = CalculateMID(pop)$ 
21:   $FPOS_{it} = |F_1|/N$ 
22:  if  $|MID_{it} - MID_{it-1}| \leq \Delta MID$  and  $FPOS_{it} = 1$  then
23:    break;
24:  end if
25: end for

```

In pRTMNSGA-III, the "p" stands for parallelization, a crucial feature for efficiently managing computational resources and distributing lengthy calculations. This parallelization is perfectly aligned with our serialized solution set evaluation approach, significantly reducing the computational overhead of the trade-off function and enhancing the overall speed of the algorithm.

We've incorporated vectorized evaluation to process sets of solutions in parallel, a change that has substantially accelerated the algorithm. Reference points play a pivotal role in structuring the search space, and they can be generated either systematically or based on user preferences. In our specific implementation, reference points are derived from pNSGA-II.

To determine the best services for a given request, pNSGA-II takes into account the service request and leverages the information about IoT services. It generates an initial service front and incorporates user preferences, ultimately providing the final non-dominated front. Similar to NSGA-III, the algorithm calculates compromises. Solutions are then allocated to reference points, building on the foundation of NSGA-III's population normalization. The algorithm's termination point is determined by a stopping criterion, which evaluates the Mean Ideal Distance (MID) and the Fraction of Pareto-optimal Solutions (FPOS) of the generated population.

It's worth noting that the solution generation function may sometimes yield inconsistent results after undergoing mutation and crossover. To address this issue, we've introduced an objective function aimed at maximizing the count of services included in a solution. This objective function effectively guides the search process towards solutions that incorporate existing services, thus enhancing the overall quality of the solutions.

One distinctive aspect of pRTMNSGA-III is its unwavering focus on Quality of Service (QoS) throughout the optimization journey. User-defined QoS preferences serve as reference points, directing the algorithm's selection towards solutions that excel in key QoS parameters such as response time, reliability, and scalability. Our fitness function, which plays a central role in the optimization process, is rooted in QoS values, effectively driving the optimization towards delivering a final output that consists of a non-dominated front showcasing solutions with finely tuned QoS characteristics. This QoS-centric methodology empowers efficient and tailored multi-cloud IoT service selection.

Furthermore, having complete knowledge of the clouds that offer a particular service allows us to define a Global ID range, which significantly aids the algorithm in generating feasible solutions. Within this context, the coding within a chromosome cell represents a service number. In each cloud, services are identified using Global IDs that correspond to the availability of a specific service, ensuring precision in the solution.

The non-dominated front, representing the best candidate solutions, is then subjected to the Fuzzy Decision Maker for the final selection of the most appropriate service set, considering various trade-offs and user preferences.

A comprehensive complexity analysis of the pRTMNSGA-III algorithm in the context of QoS-aware multi-cloud IoT service selection reveals the following insights:

-Time complexity: $O(N * t_{max} * C)$, where N represents the population size, t_{max} denotes the maximum number of iterations, and C encompasses the complexity of genetic algorithm operations.

Space complexity: $O(N * M)$, where N pertains to the population size, and M accounts for the number of objectives and the size of the solution representation.

Trade-off function complexity: $O(N * H)$, where N stands for the number of solutions, and H signifies the number of reference points.

Preference-based optimization complexity: $O(N)$, as it entails iterating through the solution set.

Hybridization and serialized evaluation complexity: Typically considered constant factors within the overall analysis.

Stopping criterion complexity: Primarily reliant on the number of iterations required for convergence, often considered a constant factor in terms of time complexity.

In sum, the time complexity of the pRTMNSGA-III algorithm is chiefly dominated by the genetic algorithm operations, generally falling into the category of $O(N * t_{max})$. The remaining complexity terms are typically treated as constants within the broader analysis.

4.3.3 Algorithm non-dominated front to Fuzzy Decision Making

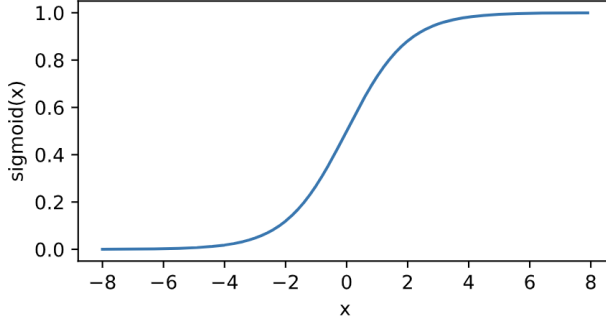
Our approach incorporates a fuzzy membership function to guide optimal solution selection based on the requester's Quality of Service (QoS) preferences.

Fuzzy decision-making is a powerful method for tackling complex multi-objective problems, enabling consideration of various criteria and fuzzy information to enhance decision-making.

To identify the best solution, we assign satisfaction levels to each objective, enabling us to find the most favorable answer while considering all objectives. Our approach employs a customized fuzzy sigmoid-type membership function, well-suited for addressing minimization problems, tailored to the specific nuances of the problem at hand.

Fuzzy sets are highly effective in representing local requirements, cost considerations, and energy efficiency in IoT service selection, offering flexibility in modeling these factors. A fuzzy decision maker identifies optimal compromises from available options, aligning them with design requirements. The choice of membership function for objectives, whether linear or nonlinear, depends on the nature of the problem.

A visual representation of this fuzzy convolution scheme is provided in Figure 1 to illustrate this concept.

**Fig. 1:** Sigmoid membership function

The value of the membership function μ_{best} , in Equation 2, indicates how much (in scale from 0 to 1) a solution is satisfying the i^{th} objective F_i . The best solution can then be selected using a fuzzy min-max proposition.

Equation 2 defines the membership function μ_{best} , quantifying the extent to which a solution satisfies a given objective (F_i), critical in selecting the best solution using a fuzzy min-max proposition.

$$\mu_{best} = Max \left\{ Min [\mu (F_j)]^k \right\} \quad (2)$$

Furthermore, Equation 3 details how we model user satisfaction over objective values, particularly in cases of objective superiority, involving variables such as U_{jm} for requester satisfaction and Q_{jmin} and Q_{jmax} for minimum and maximum satisfaction levels.

$$U_j(X, Q_{min}^j, Q_{max}^j) = \frac{1}{1 + e^{-a(X - \frac{Q_{min}^j - Q_{max}^j}{2})}} \quad (3)$$

where U_{jm} is the requester satisfaction of objective j and Q_{min}^j, Q_{max}^j is the requester minimum and maximum satisfaction of objective j .

Equation 4 calculates the average satisfaction $AS(c)$ for all objectives, incorporating satisfaction coefficients (α_i) to weigh contributions, with the composition boasting the highest average satisfaction being chosen as the compromise solution. [26]

$$AS(c) = \frac{\sum_{i=1}^{N_0} \alpha_i x U_i}{\sum_{i=1}^{N_0} \alpha_i} \quad (4)$$

Finally, the results of this fuzzy decision-making process are seamlessly integrated into databases used by composition services, ensuring that the service requester receives the most suitable set of services based on their preferences.

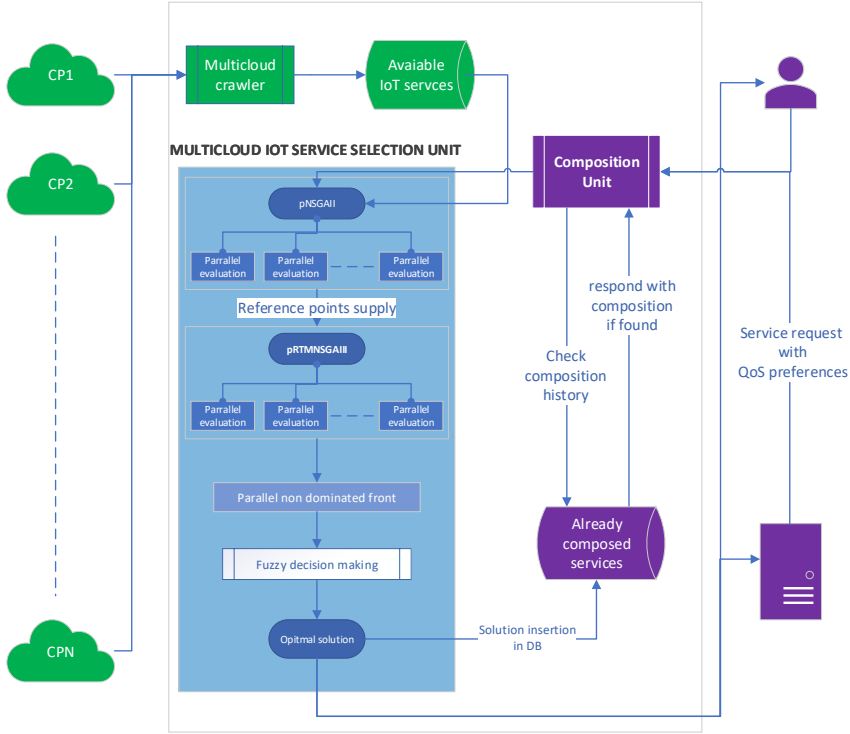


Fig. 2: Multi-cloud service composition & selection architecture

5 Experiments

To evaluate our pRTMNSGA-III algorithm, we conducted experiments to assess its viability and effectiveness. We compared it against three well-known MO algorithms: NSGA-II, RNSGA-III, and TMNSGA-III.

In addition to the three well-known MO algorithms that we will test our pRTMNSGA-III algorithm against (NSGA-II, RNSGA-III, and TMNSGA-III), we will also test it against the Non-dominated Sorting Particle Swarm Optimizer (NSPSO) [27]. NSPSO is a powerful MO algorithm that has shown remarkable performance against a set of well-known difficult test functions. It is based on the Particle Swarm Optimization (PSO) algorithm, but it has been modified to make better use of particles' personal bests and offspring for more effective non-domination comparisons.

To ensure a fair comparison of algorithm performance across various datasets, we executed each algorithm ten times on datasets with different numbers of cloud providers (5, 10, 15, 20, 25, and 30) and service counts (50, 100, 200, 300, and 500) for eight objective functions. All tests were performed on a standard Windows 10 computer with an Intel Core i7 2.60 GHz processor and

8GB of RAM, using Python 3 as the programming language. The code of this study can be found following this GitHub repository: [link](#).

We initiated all algorithms with an initial population of 2000, employing random sampling, Simulated Binary Crossover, and Polynomial Mutation with a probability of 0.9. These experiments aimed to explore the algorithm's performance limits across various service and cloud scenarios, providing insights into its comparative performance against recent techniques.

This section details dataset creation, evaluation metrics, and presents the results of our comparative experiments.

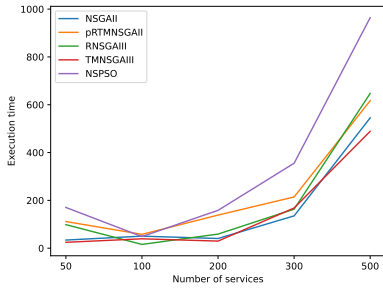
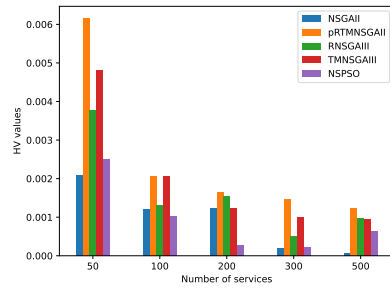
5.1 Dataset

To rigorously evaluate the performance of our pTMRSNGA-III algorithm, we generated custom evaluation datasets. Initially, we searched for suitable datasets but found none that met our requirements. Consequently, we developed a generator capable of configuring a multi-cloud environment using CP numbers and service numbers as parameters. This generator simulated the output of the cloud collector presented in [28].

In this setup, each IoT service is associated with a unique CP, and the configuration depends on the number of available services, their QoS attributes, and the overall service count. To ensure uniformity in QoS values across CPs, we carefully generated them. Subsequently, we randomly selected services to compose a CP, with the length of the service plan set randomly.

5.2 Metrics

1. **Execution Time:** Measuring the CPU time required for the algorithms to complete execution, with time units in seconds.
2. **HyperVolume (HV):** Determining the area or volume dominated by a given set of solutions concerning a reference point. Maximizing this metric is desirable.
3. **Generational Distance (GD):** Initially designed to compute the sum of distances between solutions obtained from different techniques [28], GD is intended for algorithms producing a few non-dominated points close to the Pareto front. It can also serve as a stopping criterion [29]. Minimizing GD is the objective.
4. **Generational Distance plus (GD+):** GD+ is a modified version of generational distance that evaluates a solution set's quality concerning a specified reference point set [30]. Minimizing GD+ is the goal.
5. **Inverted Generational Distance (IGD):** IGD calculates the average distance from each reference point to its nearest solution. A small IGD value indicates good convergence of solutions to the Pareto front and their uniform distribution [31].

**Fig. 3:** Execution time**Fig. 4:** HV Values

- 6. Inverted Generational Distance plus (IGD+):** IGD+ is a modified version of inverted generational distance that accounts for the Pareto dominance relationship when calculating distances [32]. Smaller IGD+ values signify better solution sets.

These performance metrics enable a comprehensive assessment of multi-objective algorithms, aiding in the selection of suitable approaches for specific optimization tasks.

5.3 Impact of the Number of Services

In order to compare the four resolution algorithms' performances, we will investigate how the variation in the number of services affects them. Table 5 below lists all the outcomes of numerous iterations on various datasets with 50, 100, 200, 300, and 500 services each, with the number of clouds set to 20.

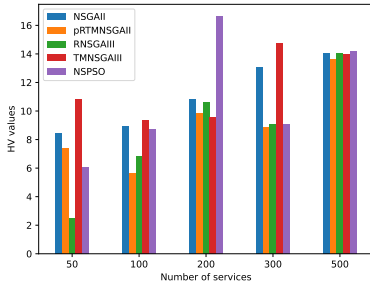
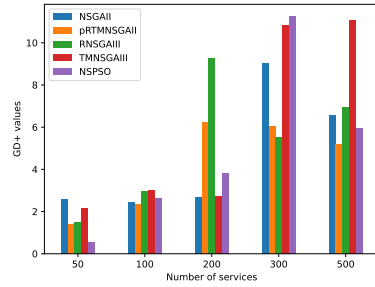
In the line graph presented in Figure 3, the average execution time is visually represented for four experimented algorithms: NSGA-II, pRTMNSGA-III, RNSGA-III, and TMNSGA-III. It is evident that NSGA-II and TMNSGA-III exhibit nearly constant and significant growth in their running time values until they peak at a service count of 500. In contrast, the remaining two algorithms show a somewhat inconsistent pattern in running times, with a significant increase occurring between 50 and 300 services, followed by another peak at 500 services.

Notably, the NSPSO algorithm consistently exhibits longer execution times than all other algorithms across different service counts. Overall, the pRTMNSGA-III algorithm exhibits standard running times similar to the other variants in the same environment.

Moving to Figure 4, it becomes apparent that as the number of services increases, the HV values of all four algorithms decrease. Notably, pRTMNSGA-III achieves the highest HV value, starting at 0.006 when the number of services is 50 and decreasing to almost 0.002 when the number of services reaches 500. A similar pattern is observed for the other algorithms, albeit with lower performance levels.

Table 5: Metrics values varying number of services

	nombre de services	NSGAII	pRTMNS-GAII	RNSGAIII	TMNSGAIII	NSGAIII	pNSGAII	NSPSO
HV	50	0.0021	0.00616	0.00378	0.00481	0.00444528	0.0022869	0.0024948
	100	0.001213	0.00207	0.001299	0.00205	0.00223558	0.00138039	0.00101972
	200	0.001227	0.001656	0.001534	0.0012256	0.00101704	0.00060246	0.00026862
	300	0.000202	0.00147	0.00051	0.001	0.00035394	0.00030926	0.00021318
GD	500	0.0000569	0.00123	0.00098	0.00095	0.00082418	2.2532E-05	0.00062414
	50	8.40844575	7.39958073	2.46533558	10.7912239	3.59445927	7.24808023	6.05347589
	100	8.93278089	5.6439298	6.80146519	9.38068557	7.39319266	9.11426526	8.75271089
	200	10.8489333	9.4497606	10.5929412	9.59524531	14.2475059	11.4903786	16.6422636
	300	13.0911133	8.83130847	9.06153298	14.7510961	9.65959416	9.56666351	9.04809436
	500	14.0647026	13.6238777	14.0418036	13.9660573	16.230998	16.981717	14.1645581
GD+	50	2.60144333	1.37906064	1.5	2.15920193	0.63	3.20237673	0.53069444
	100	2.43653866	2.32399757	2.9519944	2.989	4.92392665	2.61196944	2.6389752
	200	2.67867412	6.21736792	9.24860766	2.74418174	12.2544052	4.49481517	3.83757045
	300	9.00766114	6.04076543	5.5	10.8511189	8.657	7.38628214	11.2595764
	500	6.56228624	5.17213278	6.92744328	11.0464759	5.30642155	5.19330091	5.93737174
	50	20.9528134	6.17438492	10.7974493	11.063472	16.5632873	6.30679684	13.961102
IGD	100	20.9799253	20.508043	14.3110194	14.5103573	10.5901543	18.9868324	12.7995056
	200	32.9540136	24.6710538	19.759611	25.0965694	25.7270135	28.4644424	29.2079787
	300	39.2982429	23.3832585	32.4306639	24.7754154	29.5767655	38.9992976	39.4356873
	500	55.6083465	26.5262342	45.444498	30.3131377	9.17977885	61.614048	27.5870021
IGD+	50	39.7654383	20.8543625	21.4846353	27.4512796	30.3363051	41.4753522	26.4690707
	100	18.0873392	13.2833285	12.3066568	14.0340484	7.05171437	4.75697022	21.302823
	200	26.1614918	24.4282201	23.6533586	22.8678909	23.416825	23.6499886	26.4337135
	300	20.9260493	17.1187488	20.8286469	17.2858309	22.6823965	23.1183611	15.871429
	500	18.6220786	11.1024641	12.1788747	17.6409645	2.67935244	22.193827	19.0964756

**Fig. 5: GD Values****Fig. 6: GD+ Values**

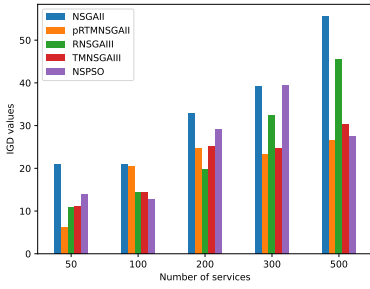
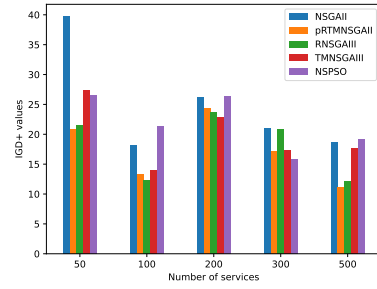
The graph in Figure 5 illustrates that, to varying degrees, the GD increases as the number of services increases for all the algorithms. For example, RNSGA-III starts with a very low GD but experiences a sharp increase as the number of services grows. In contrast, our method consistently maintains the lowest GD values.

Furthermore, when considering GD+ as a metric to minimize, pRTMNSGA-III consistently yields lower GD+ values compared to the other algorithms. This makes it the preferred algorithm. The graph in Figure 6 displays the GD+ values for the tested algorithms as the number of services varies, showing that GD+ values increase in tandem with the increase in services, where pRTMNSGA-III gave generally lower GD+ values in contrast to NSPSO where it couldn't keep the smaller values for all tests.

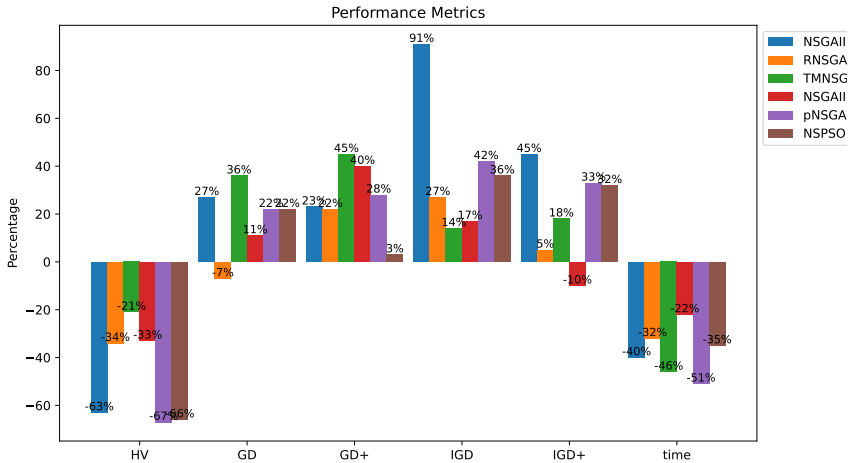
In Figure 7, the four algorithms demonstrate compatibility in terms of IGD values as the number of services steadily increases. Among these algorithms, NSGA-II exhibits the least compatibility, while our proposed pRTMNSGA-III stands out as the most adaptable, especially when dealing with more than 200 services. In these scenarios, a noticeable decline in IGD values is observed, with pRTMNSGA-III consistently displaying lower IGD values. Consequently, pRTMNSGA-III emerges as the top-performing algorithm in terms of this metric.

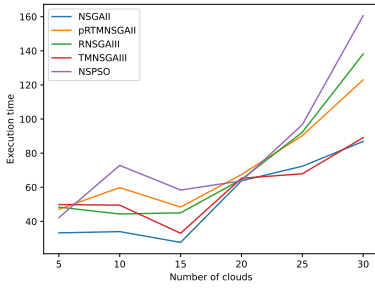
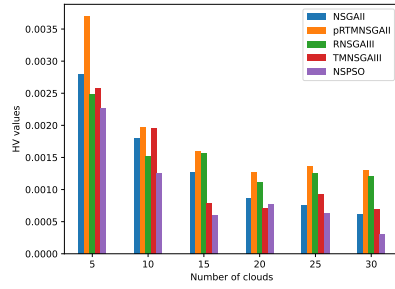
Likewise, in Figure 8, we can observe that the NSGA-II, pRTMNSGA-III, RNSGA-III, and TMNSGA-III algorithms initially exhibit higher IGD values, which then irregularly decline as the number of services increases, while NSPSO is getting mostly the higher IGD+ values. Throughout these variations, pRTMNSGA-III consistently maintains lower IGD values, underscoring its superior capabilities in managing this aspect in comparison to the other algorithms. It is noteworthy that the NSPSO algorithm consistently lags behind another algorithm in all scenarios across different service counts, further emphasizing the robust performance of our pRTMNSGA-III approach.

The accompanying Figure 9 illustrates the results. Concerning hypervolume (HV), pRTMNSGA-III demonstrates significant improvements over the

**Fig. 7:** IGD Values**Fig. 8:** IGD+ Values

other algorithms, with a more than 60% advantage over NSGA-II and NSPSO, a 34% advantage over RNSGA-III, and a 21% advantage over TMNSGA-III. In terms of generational distance (GD), generational distance plus (GD+), and inverted generational distance (IGD), pRTMNSGA-III maintains a competitive edge, outperforming one or more algorithms across these metrics. Notably, for IGD, pRTMNSGA-III exhibits an impressive 91% improvement over NSGA-II. Additionally, pRTMNSGA-III showcases its efficiency by having a 40% lower execution time than NSGA-II, a 32% lower time than RNSGA-III, and a 46% lower time than TMNSGA-III. despite NSPSO gave lower execution time of 35% in comparison with our proposed approach

**Fig. 9:** pRTMNSGA-III Performance Comparison when varying number of services

**Fig. 10:** Execution time**Fig. 11:** HV Values

5.4 Impact of the Number of Clouds

The effectiveness of the four resolution algorithms will be compared based on how they perform a different facing number of clouds. The results from numerous executions on various datasets with 5, 10, 15, 20, 25, and 30 clouds each, with the number of services set to 200, are shown in Table 6 below.

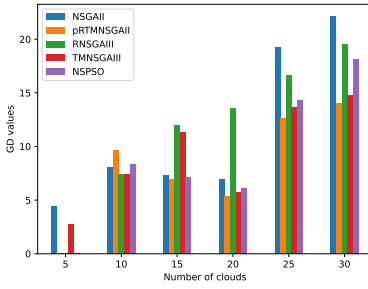
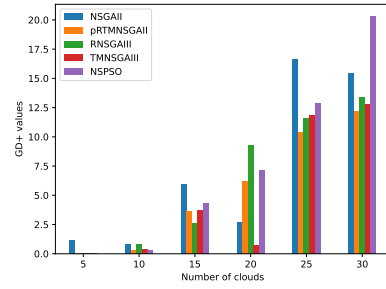
Figure 10, much like Figure 3, provides insight into the execution times of the four compared algorithms concerning the number of clouds. Significantly, both NSGA-II and TMNSGA-III exhibit a pronounced increase in runtime at the 15-cloud mark, followed by a sharp rise that can extend to nearly 70 seconds when reaching 20 clouds. This escalation is followed by a sustained upward trajectory. Conversely, RNSGA-III maintains relatively stable execution times as the number of clouds increases, punctuated by a noticeable spike that reaches up to 140 seconds, making it the algorithm with the longest execution time. In contrast, pRTMNSGA-III undergoes a significant runtime increase at the ten-cloud mark, succeeded by a rapid drop to 50 seconds, and an immediate subsequent increase, ultimately peaking at 120 seconds. **Remarkably, the NSPSO algorithm continues to exhibit suboptimal performance in terms of execution time, consistently underperforming when compared to the other algorithms, even as the number of clouds increases.**

Figure 11 sheds light on the challenge of achieving higher HV values as the number of clouds increases. Remarkably, the proposed pRTMNSGA-III surpasses the other algorithms consistently, attaining the highest HV scores. These HV scores range from 0.0035 for scenarios with five clouds to 0.0014 when dealing with 30 clouds. This underscores the complexity and difficulty of obtaining high-quality solutions in scenarios involving a larger number of clouds.

Figure 12 depicts an increase in GD values for RNSGA-III and TMNSGA-III and NSPSO, especially when a larger number of clouds is considered. In contrast, GD values for NSGA-II and pRTMNSGA-III exhibit slight fluctuations before ascending with the growing number of clouds. Notably, pRTMNSGA-III appears to be the most suitable algorithm in terms of the GD

Table 6: Metrics values varying number of cloud

	nombre cloud	NSGAI	pRTMNSGAI	RNSGAI	TMNSGAI	NSGAI	pNSGAI	NSPSO
HV	5	0.002801	0.0037	0.00249	0.00258	0.00339387	0.00473649	0.00226321
	10	0.001801	0.001978	0.001519	0.001961	0.00183647	0.00243495	0.0012535
	15	0.00127	0.00159	0.00157	0.00079	0.002826	0.0005588	0.00060071
	20	0.000863	0.001262	0.00111	0.0007115	0.00181707	0.00077325	0.00076721
	25	0.0007617	0.001358	0.001258	0.0009299	0.00129448	0.00097802	0.00062383
	30	0.00061	0.001295	0.0012031	0.00070047	0.00138597	0.00044103	0.0003111
GD	5	4.38817907	0	0	2.76936054	0	4.60657837	0
	10	8.03298118	9.61635343	7.39336078	7.38470297	7.14937988	8.32195619	8.34710432
	15	7.32988386	6.96923362	12.0079772	11.3000583	9.94260514	7.723581	7.1687624
	20	6.95966737	5.33185892	13.6087892	5.74314026	20.984753	5.34064034	6.15117272
	25	19.2188906	12.5952418	16.6630276	13.6706758	8.09823139	17.208937	14.2635516
	30	22.1463176	14.0327769	19.5567086	14.7317583	28.6505781	20.6064341	18.0886538
GD+	5	1.14277146	0	0	0.07194137	0	1.58730956	0
	10	0.77451275	0.31458886	0.81399499	0.42643981	0.51444483	0.81865998	0.33943591
	15	5.97849737	3.61635343	2.59241969	3.72137578	3.55939224	5.58353129	4.3578575
	20	2.67867412	6.21736792	9.24860766	0.74418174	6.24281017	0.57055759	7.10293068
	25	16.6685081	10.3531676	11.5955823	11.8778166	14.8307497	19.4854859	12.8363096
	30	15.4240337	12.1874433	13.3490315	12.791444	18.2214281	22.1489123	20.317226
IGD	5	20.23821	15.5095192	26.2707375	16.4190193	15.4997351	18.8440978	19.6275773
	10	10.8929228	10.4428666	28.2537422	15.0781334	23.9874271	13.372063	10.3155979
	15	13.1239568	22.5000663	23.8961929	21.2726329	23.418269	12.2732514	23.3081473
	20	16.2536544	17.2078095	39.2468474	17.1241704	22.4491967	19.8659682	23.9903939
	25	26.0837349	25.0086822	41.6268095	28.0081443	63.7306453	27.5183404	25.483809
	30	28.4456623	26.0810265	42.2873405	29.6337327	22.4968651	30.0928114	27.9336404
IGD+	5	3.20967499	4.57932967	10.9845623	4.64255821	7.8649466	3.48570703	5.26386698
	10	5.53219519	5.20076466	19.5610178	5.9782752	35.1902711	7.28036887	4.73002689
	15	7.37224636	8.65773322	15.7822716	10.0550525	27.5558462	4.85831035	10.5423123
	20	15.6652364	14.5243343	23.8310703	14.8432757	15.2757161	23.3412023	8.97618047
	25	22.0420118	12.7363441	34.3882876	13.6602872	52.0638674	30.2636822	29.7787579
	30	21.3600653	20.4627761	31.9238663	21.0847762	22.1870871	35.692669	33.6063616

**Fig. 12: GD Values****Fig. 13: GD+ Values**

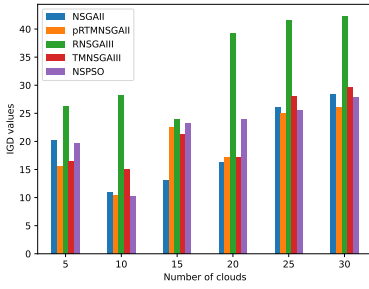
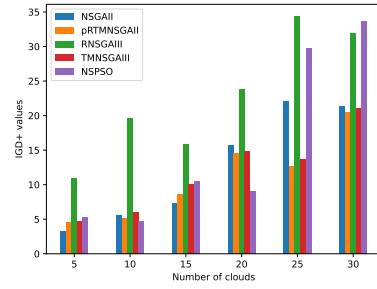
metric, while the other algorithms initiate with lower values, ascend to nearly 10 when the cloud count reaches 10, and then exhibit erratic developments with high GD scores. This is particularly evident for NSGA-II and RNSGA-III.

Figure 13 illustrates how GD plus values steadily increase for all algorithms as the number of clouds rises. However, an exception occurs when the cloud count reaches 20, where NSGA-II, NSPSO and TMNSGA-III exhibit better performance. Throughout this variation, pRTMNSGA-III consistently maintains relatively low GD plus values compared to the other algorithms.

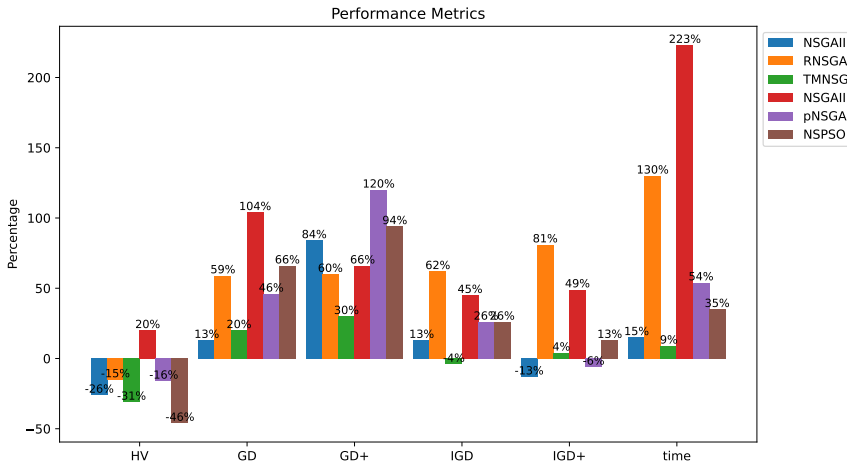
Figure 14 demonstrates that all algorithms experience erratic increases in their IGD values as the number of clouds increases. NSPSO gave better results in higher numbers of clouds, and notably, NSGA-II and pRTMNSGA-III stand out as competitors, consistently achieving commendable and remarkably similar IGD values. On the other hand, RNSGA-III unquestionably performs the worst in terms of IGD values.

Figure 15 presents the IGD plus values of the different algorithms, showcasing how these values increase with an expanding cloud count. In cases where the number of clouds ranges from 5 to 15, NSGA-II and NSPSO may provide better IGD plus values due to the low number of clouds and the algorithm's simplicity, which predisposes it to yield better results. However, the algorithm's performance deteriorates, falling behind both RNSGA-III and pRTMNSGA-III when dealing with a larger number of clouds. Consequently, pRTMNSGA-III and RNSGA-III produce the most favorable IGD plus values, with a slight edge in favor of the proposed pRTMNSGA-III.

In our comparative analysis Figure 16, we assessed the performance of pRTMNSGA-III against six prominent multi-objective optimization algorithms: NSGA-II, RNSGA-III, TMNSGA-III, NSGAIII, pNSGAII and NSPSO. The results, as shown in the table, reveal several noteworthy trends. In terms of hypervolume (HV), pRTMNSGA-III exhibits a 26% improvement over NSGA-II, a 15% improvement over RNSGA-III, a 31% improvement over TMNSGA-III, highlighting its superior ability to explore solution spaces. Additionally, pRTMNSGA-III demonstrates competitive performance in terms of generational distance (GD), generational distance plus (GD+), and inverted generational distance (IGD), outperforming one

**Fig. 14:** IGD Values**Fig. 15:** IGD plus Value

or more algorithms across these metrics. Furthermore, in terms of computational efficiency, pRTMNSGA-III is notable for its relatively lower execution time, showcasing a 15% improvement over NSGA-II, a 130% improvement over RNSGA-III, and a 9% improvement over TMNSGA-III. These findings collectively underscore the effectiveness of pRTMNSGA-III in multi-objective optimization tasks, making it a promising choice for a wide range of applications.

**Fig. 16:** pRTMNSGA-III Performance Comparison when varying number of clouds

5.5 Performance Comparison of RTMNSGA-III and pRTMNSGA-III

Figures 17 and 18 provide a comprehensive performance assessment of pRTMNSGA-III. In 18, the consistent computational efficiency of pRTMNSGA-III across varying numbers of clouds is evident, outperforming RTMNSGA-III. Simultaneously, in Figure 17, it showcases its versatility in handling diverse numbers of services while maintaining efficiency. These results underscore the adaptability and effectiveness of pRTMNSGA-III.

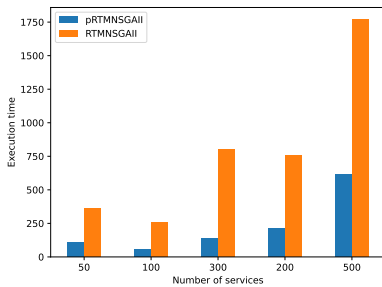


Fig. 17: Execution time varying number of services

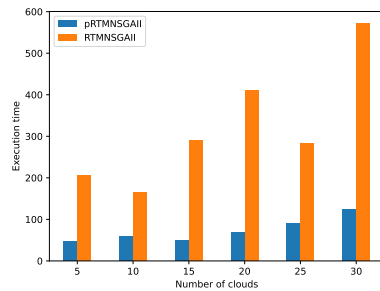


Fig. 18: Execution time varying number of clouds

5.6 Pareto-Front Analysis and discussion

After testing all six algorithms (NSGA-II, RNSGA-III, TMNSGA-III, NSGAIII, pNSGAII, NSPSO), as indicated in Table 7, we ranked the algorithms from 1 to 7 based on key performance indicators (KPIs), including GD, GD+, IGD, IGD+, HV, and execution time. We then compared the best-performing algorithm in each case with our hybridization approach using the Pareto front.

Table 7: Ranking of the compared algorithms.

	NSGAII	pRTMNSGAII	RNSGAIII	TMNSGAIII	NSGAIII	pNSGAII	NSPSO
Ranking (#Cloud variation)	143	98	183	137	147	145	167
Ranking (#Services variation)	151	81	100	111	115	132	150
General Ranking	294	179	283	248	262	277	317

The scatter plots provide a comparison between our algorithm and the highest-ranked algorithm among the three tested (pRTMNSGA-III and TMNSGA-III, according to the Metrics model). Each algorithm is represented by a different color and is evaluated across all 2 by 2 combinations of QoS

parameters, creating 2D scatter plots of the Pareto fronts. The proximity of the plots to the axes indicates the quality of the solutions. In this context, our algorithm, pRTMNSGA-III, is depicted in green, while the TMNSGA-III algorithm is represented in pink.

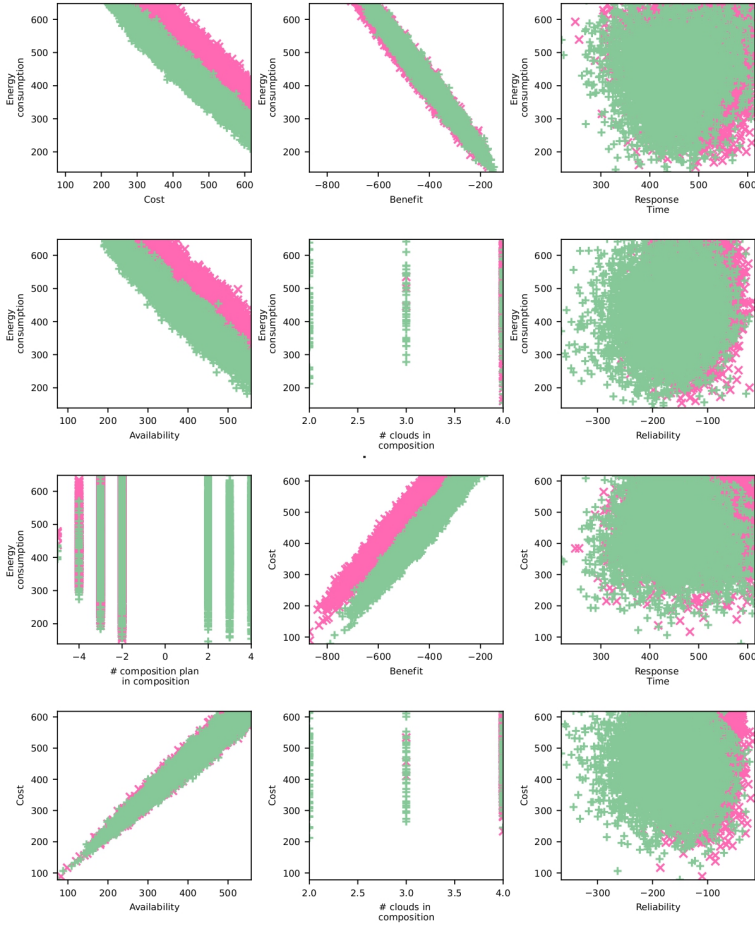


Fig. 19: The QoS values of solutions obtained by pRTMNSGA-III and TMNSGA-III for service selection in dataset of 200 services and 20 clouds (Part 1).

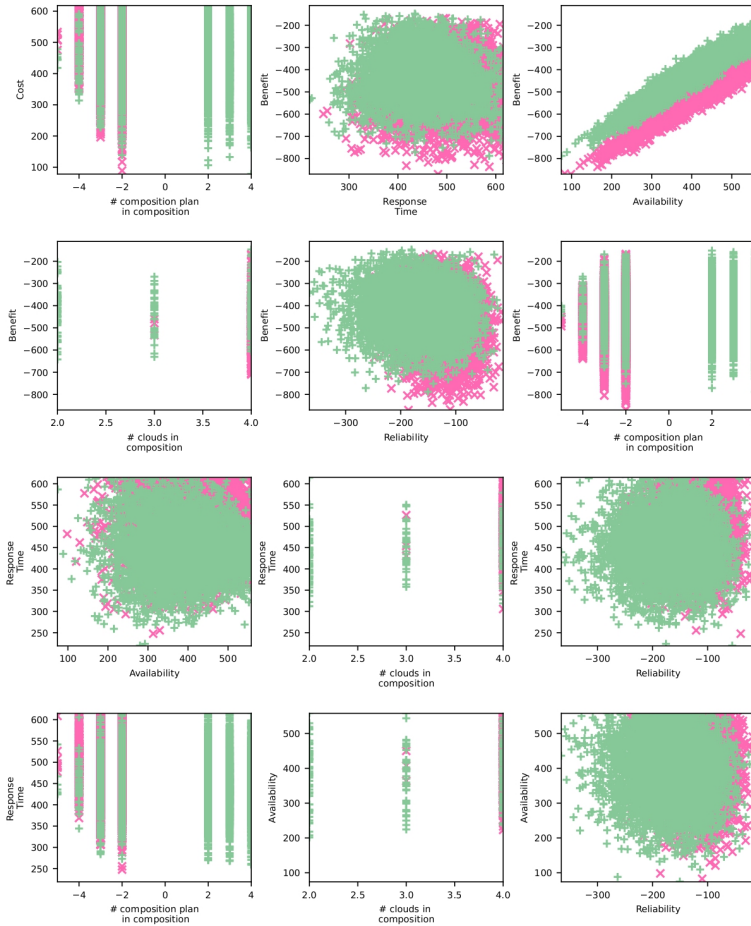


Fig. 20: The QoS values of solutions obtained by pRTMNSGA-III and TMNSGA-III for service selection in dataset of 200 services and 20 clouds (Part 2).

The graphs reveal a notable consistency and alignment of solutions due to the numerical nature of QoS values and variations in the number of composition plans and included clouds. Notably, our algorithm (depicted in green)

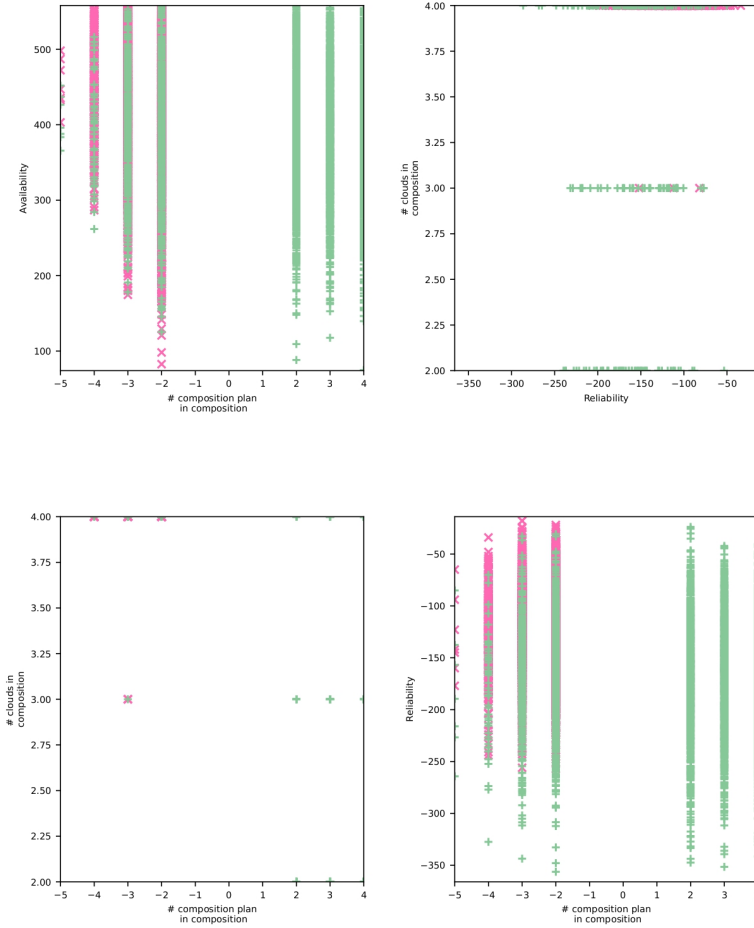


Fig. 21: The QoS values of solutions obtained by pRTMNSGA-III and TMNSGA-III for service selection in dataset of 200 services and 20 clouds (Part 3).

consistently positions its solutions near ($X=0$, $Y=0$) in figures evaluating algorithm performance across different QoS parameters. This pattern suggests the algorithm's superiority compared to the other three algorithms.

Based on the aforementioned observations, the multi-objective algorithm pRTMNSGA-III outperforms alternative approaches by consistently meeting

QoS standards and delivering efficient Pareto solutions across various QoS metrics. To summarize, this study highlights the challenges faced by the tested NSGA-II, RNSGA-III, and TMNSGA-III algorithms, particularly in unfavorable scenarios. In contrast, pRTMNSGA-III excels in producing remarkable outcomes even in challenging situations. Notably, the NSPSO algorithm did not exhibit great execution times, nor did it achieve significant solution quality metrics, as seen in previous studies [16], have examined PSO variant and found that they did not achieve great execution times and did not yield significant metric values. The pRTMNSGA-III algorithm demonstrates its superiority through the highest Pareto efficiency metrics, including IG, IGD, IG+, and IGD+. This clear performance advantage underscores the limitations of NSGA-II algorithms in distinguishing between solutions based on their compromise, potentially explaining their subpar results. An increased focus on high-tradeoff options can facilitate the discovery of more tailored trade-off solutions in regions of interest. For example, RNSGA-III tends to struggle with generating Pareto optimal solutions when using the Das and Dennis benchmarks on the unit simplex. Furthermore, the algorithm yields better HV results than its counterparts, highlighting the quality of a non-dominant ensemble of approximations. In contrast, both NSGA-II and RNSGA-III are known to encounter challenges in creating solutions with sufficient diversity, particularly for certain problem types. Additionally, the naive stopping criterion employed by NSGA-II and RNSGA-III algorithms contributes to difficulties in assigning solutions to benchmarks. A potential solution involves parallelizing the evaluation process to alleviate computational demands and enhance performance.

The key novelties of pRTMNSGA-III can be summarized as follows:

Hybridization of RNSGA-III and TMNSGA-III: pRTMNSGA-III leverages the strengths of both algorithms, which individually offer improvements over NSGA-III. This hybridization is unique in the context of Multi-Cloud IoT Service Selection.

Parallelization: To handle the extensive array of provided services in multi-cloud environments efficiently, pRTMNSGA-III introduces parallelization. This approach enhances the algorithm's performance and scalability.

Serialized Evaluation: The proposed serialized evaluation of a solution set optimizes the evaluation process, reducing trade-off function expenses and speeding up the algorithm.

Incorporation of User Preferences: pRTMNSGA-III is designed to work with user preferences from the outset. It employs trade-off functions and a novel stopping criterion based on MID and FPOS to facilitate efficient preference-based optimization.

Enhanced Solution Selection: The algorithm addresses the challenge of generating consistent solutions aligned with user preferences by incorporating an objective function to maximize the number of services in a solution.

Combination of Features: pRTMNSGA-III combines several advanced features, including reference points, trade-off functions, parallelization, and

preference-based optimization, into a unified framework, making it a comprehensive solution for the Multi-Cloud IoT Service Selection problem.

Efficiency and Speed: The primary focus of pRTMNSGA-III is to provide a substantial reduction in computation time while maintaining higher solution diversity and optimizing solution density in trade-off areas.

pRTMNSGA-III offers a novel and efficient approach to Multi-Cloud IoT Service Selection by integrating the strengths of existing algorithms and introducing innovative features. This approach is poised to significantly enhance the quality and speed of service selection in complex multi-cloud environments.

6 Conclusion

In this research, we proposed a novel multi-objective algorithm named pRTMNSGA-III. This algorithm is designed to offer optimal results for selecting services that fulfill user requirements by leveraging the strengths of two algorithms, RNSGA-III and TMNSGA-III. The proposed algorithm combines the best aspects of each and employs parallel evaluation of solutions to improve its efficiency. Moreover, a more suitable fuzzy decision-making approach is utilized to select the best solution for the service requester.

To evaluate the performance of the proposed algorithm, we conducted multiple experiments and compared it with other existing algorithms for the same problem. Results demonstrate the effectiveness and superiority of pRTMNSGA-III over other tested algorithms, with improved non-dominated edges of up to 37% over NSGA-II, 36% over RNSGA-III, and 30% over TMNSGA-II.

In summary, the proposed pRTMNSGA-III algorithm outperforms existing state-of-the-art algorithms for QoS-aware service selection in multi-cloud IoT environments, as demonstrated by the experimental results presented in the abstract. Future work includes testing the algorithm in a GPU environment to enable real parallel computing, thus resolving the issue of execution time. Furthermore, we plan to investigate the use of deep reinforcement learning techniques for optimizing this particular problem, given their success in many optimization problems.

Author contributions

Ahmed ZBOUCHI: Conceptualization, Methodology, Software, Experimentation, Formal analysis, Investigation, Writing, and editing.
Youssef AKLOUF: Methodology, Resources, Supervision.

Conflict of interest

The authors declare no potential conflict of interest.

Data availability

Data sharing does not apply to this article. The used datasets are randomly generated during the execution. All the necessary details to generate them are given in this paper.

References

- [1] Vailshery, L.S.: Global IOT connected devices by technology 2030 (2022). <https://www.statista.com/statistics/1183463/iot-connected-devices-worldwide-by-technology/>
- [2] Aldawsari, B., Baker, T., England, D.: Trusted energy-efficient cloud-based services brokerage platform. *International Journal of Intelligent Computing Research* **6**(4), 630–639 (2015). <https://doi.org/10.20533/ijicr.2042.4655.2015.0078>
- [3] Sefati, S.S., Halunga, S.: A hybrid service selection and composition for cloud computing using the adaptive penalty function in genetic and artificial bee colony algorithm. *Sensors* **22**(13) (2022). <https://doi.org/10.3390/s22134873>
- [4] Thakur, N., Singh, A., Sangal, A.L.: Cloud services selection: A systematic review and future research directions. *Computer Science Review* **46**, 100514 (2022). <https://doi.org/10.1016/j.cosrev.2022.100514>
- [5] Ghazali, T.E.: *Metaheuristics: From Design to Implementation*. Wiley, ??? (2009). <https://doi.org/10.5555/1718024>
- [6] Mohamed, A.M., Abdelsalam, H.M.: A multicriteria optimization model for cloud service provider selection in multicloud environments. *Software: Practice and Experience* **50**(6), 925–947 (2020)
- [7] Yang, L., Wang, L., Luo, T., Zhang, H., Liu, J., Xing, J.: Towards iot-enabled dynamic service optimal selection in multiple manufacturing clouds. *Journal of Manufacturing Systems* **56**, 213–226 (2020)
- [8] Baker, T., Aldawsari, B., Asim, M., Tawfik, H., Buyya, R.: An energy-aware service composition algorithm for cloud-based iot applications. *Journal of Cloud Computing* **6**(1), 1–11 (2017)
- [9] Chen, Y., Shen, W., Wang, X., Lin, T., Xiao, Y.: Iot-enabled dynamic service selection across multiple manufacturing clouds. 2016 IEEE 20th International Conference on Computer Supported Cooperative Work in Design (CSCWD), 676–681 (2016)
- [10] Ait Aissa, M., Abdelsalam, H.M.: Energy-centered and qos-aware services selection for internet of things. *IEEE Transactions on Automation Science*

- and Engineering **13**(3), 1256–1269 (2016)
- [11] Benouiza, A., Boukaâbache, M., Boughanem, M.: A genetic algorithm-based approach for fluctuating qos aware selection of iot services. *IEEE Internet of Things Journal* **9**(20), 16403–16415 (2022)
 - [12] Wang, R., Lu, J.: Qos-aware service discovery and selection management for cloud-edge computing using a hybrid meta-heuristic algorithm in iot. *Wireless Personal Communications* **112**(1), 1–22 (2022)
 - [13] Mohamed, A.M., Abdelsalam, H.M.: Multicuckoo: Multi-cloud service composition using a cuckoo-inspired algorithm for the internet of things applications. *IEEE Access* **6**, 56737–56749 (2018)
 - [14] Kumrai, T., Ota, K., Dong, M., Kishigami, J., Sung, D.K.: Multiobjective optimization in cloud brokering systems for connected internet of things. *IEEE Journal of the Internet of Things* **8**(16), 12426–12437 (2021)
 - [15] Liu, J., Gu, X., Fu, X., Zhang, H., Buyya, R., Vo, H.V.: A new multi-objective evolutionary algorithm for inter-cloud service composition. *IEEE Transactions on Cloud Computing* **6**(1), 59–72 (2018)
-
- [17] Rice, O., Smith, R.E., Nyman, R.: Parallel multi-objective genetic algorithm. In: Dediu, A.-H., Martín-Vide, C., Truthe, B., Vega-Rodríguez, M.A. (eds.) *Theory and Practice of Natural Computing*, pp. 217–227. Springer, Berlin, Heidelberg (2013)
 - [18] Deb, K.: *Evolutionary Multi-criterion Optimization: 10th International Conference, Emo 2019, East Lansing, MI, USA, March 10–13, 2019: Proceedings*. Springer, ??? (2019). <https://doi.org/10.1007/978-3-030-12598-1>
 - [19] Vesikar, Y., Deb, K., Blank, J.: Reference point based nsga-iii for preferred solutions. 2018 IEEE Symposium Series on Computational Intelligence (SSCI) (2018). <https://doi.org/10.1109/ssci.2018.8628819>
 - [20] DE BUCK, V.: *Improving evolutionary algorithms for multi-objective optimisation*. PhD thesis (2017)
 - [21] A new multi-objective evolutionary algorithm for inter-cloud service composition. *KSII Transactions on Internet and Information Systems* **12**(1) (2018). <https://doi.org/10.3837/tiis.2018.01.001>

- [22] Chauhan, S.S., Pilli, E.S., Joshi, R.C., Singh, G., Govil, M.C.: Brokering in interconnected cloud computing environments: A survey. *Journal of Parallel and Distributed Computing* **133**, 193–209 (2019). <https://doi.org/10.1016/j.jpdc.2018.08.001>
- [23] Zhang, X., Geng, J., Ma, J., Liu, H., Niu, S.: A qos-driven service selection optimization algorithm for internet of things (2020). <https://doi.org/10.21203/rs.3.rs-69961/v1>
- [24] Vesikar, Y., Deb, K., Blank, J.: Reference point based nsga-iii for preferred solutions. 2018 IEEE Symposium Series on Computational Intelligence (SSCI) (2018). <https://doi.org/10.1109/ssci.2018.8628819>
- [25] Hashem, I., Telen, D., Nimmegeers, P., Logist, F., Van Impe, J.: A novel algorithm for fast representation of a pareto front with adaptive resolution: Application to multi-objective optimization of a chemical reactor. *Computers & Chemical Engineering* **106**, 544–558 (2017). <https://doi.org/10.1016/j.compchemeng.2017.06.020>
- [26] Moeini-Aghaie, M., Abbaspour, A., Fotuhi-Firuzabad, M.: Incorporating large-scale distant wind farms in probabilistic transmission expansion planning—part i: Theory and algorithm. *IEEE Transactions on Power Systems* **27**(3), 1585–1593 (2012). <https://doi.org/10.1109/tpwrs.2011.2182363>
- [27] Li, X.: A non-dominated sorting particle swarm optimizer for multiobjective optimization. In: Cantú-Paz, E., Foster, J.A., Deb, K., Davis, L.D., Roy, R., O'Reilly, U.-M., Beyer, H.-G., Standish, R., Kendall, G., Wilson, S., Harman, M., Wegener, J., Dasgupta, D., Potter, M.A., Schultz, A.C., Dowsland, K.A., Jonoska, N., Miller, J. (eds.) *Genetic and Evolutionary Computation — GECCO 2003*, pp. 37–48. Springer, Berlin, Heidelberg (2003)
- [28] Liu, Y., Wei, J., Li, X., Li, M.: Generational distance indicator-based evolutionary algorithm with an improved niching method for many-objective optimization problems. *IEEE Access* **7**, 63881–63891 (2019). <https://doi.org/10.1109/access.2019.2916634>
- [29] Audet, C., Bignon, J., Cartier, D., Le Digabel, S., Salomon, L.: Performance indicators in multiobjective optimization. *European Journal of Operational Research* **292**(2), 397–422 (2021). <https://doi.org/10.1016/j.ejor.2020.11.016>
- [30] Ishibuchi, H., Masuda, H., Tanigaki, Y., Nojima, Y.: Modified distance calculation in generational distance and inverted generational distance. *Lecture Notes in Computer Science*, 110–125 (2015). https://doi.org/10.1007/978-3-319-15892-1_8

- [31] Fonseca, C.M., Paquete, L., Lopez-Ibanez, M.: An improved dimension-sweep algorithm for the hypervolume indicator. 2006 IEEE International Conference on Evolutionary Computation. <https://doi.org/10.1109/cec.2006.1688440>
- [32] Ishibuchi, H., Masuda, H., Nojima, Y.: A study on performance evaluation ability of a modified inverted generational distance indicator. Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation (2015). <https://doi.org/10.1145/2739480.2754792>